

SYSTEM ARCHITECTURE ANALYSIS

Women's Health & Menstrual Cycle Tracking System

A cloud-hosted, web & mobile healthcare platform for cycle tracking, symptom monitoring, and predictive insights — analyzed against an existing workplace system.

Custom Auth

Managed DB

Push Notifications

Web + Mobile

Student Submission Form

SE ZG651 / SS ZG653 — Software Architectures | Situated Learning Assignment (EC-1)

NAME	Sarankirthic SP
ID NO.	2026NS93107
INDUSTRY / DOMAIN	Healthcare Technology
ORGANISATION TYPE	Startup

***Instructor prompt:** Select an existing workplace system to document its goal, requirements, ASR utility tree, architectural tactics, and 4 core architectural diagrams.*

System Purpose & Primary Goal

Purpose A cloud-hosted healthcare application, accessible via web and mobile clients, that enables users to track menstrual cycles, record symptoms and health observations, and receive personalized cycle-related insights and reminders.

1

Track

Track menstrual cycle history

2

Monitor

Monitor symptoms and health patterns

3

Predict

Predict upcoming menstrual cycles

4

Remind

Deliver reminders via push notification

5

Review

Allow users to view historical health information

6

Control

Allow users to export or delete their data on request

Functional Requirements

User Management

- Registration & login via custom auth service
- Session management
- User profile management
- Password / account recovery

Cycle Tracking

- Record menstrual start & end dates
- View current & previous cycles
- Calculate cycle duration
- Maintain cycle history

Symptom Tracking

- Record symptoms & health observations
- Track symptoms against cycle dates
- View historical symptom patterns

Prediction & Insights

- Predict upcoming cycles from historical data
- Provide cycle-related insights
- Identify recurring patterns

Notifications

- Period reminders via mobile push
- User-configurable reminder timing

Data Rights

- Export personal health data
- Delete account & associated health data

Non-Functional Requirements

Grounded in real platform choices — not fabricated numbers.

Quality Attribute	Requirement	Note on Basis
Security	Protect health data in transit via HTTPS; leverage the managed database's built-in encryption-at-rest.	<i>Platform capability, not custom-built.</i>
Privacy / Compliance	Restrict access to identifiable health data; support user-initiated export and deletion.	<i>Mirrors the Data Rights functional requirements.</i>
Availability	Core recording (cycle/symptom entry) stays usable even if prediction is degraded.	<i>Design goal, not an uptime claim.</i>
Performance	Cycle history should feel responsive on both web and mobile clients.	<i>Left qualitative — no fabricated response times.</i>
Scalability	System accommodates a growing user base without a full redesign.	<i>Reflects the cloud deployment model.</i>
Reliability	Recorded cycle & symptom data is not lost once saved.	<i>Relies on managed DB durability guarantees.</i>
Usability	Minimal effort to record cycle/symptom info on either client.	<i>Ties to the dual web/mobile requirement.</i>
Modifiability	New health-tracking features addable without reworking unrelated modules.	<i>Supports module decomposition (Slide 7).</i>

ASR Utility Tree

Six candidate ASRs, each traced to a concrete stimulus/response, prioritized by importance & difficulty. Top 5 carried forward.

ASR	Stimulus	Response	Imp.	Diff.	Top 5?
Security	Unauthenticated request attempts to access another user's health records.	Custom auth layer rejects the request before it reaches health data.	H	M	✓
Privacy	A user requests deletion of their account.	Associated health records removed as part of the deletion workflow.	H	M	✓
Availability	The prediction function is unavailable or slow.	Cycle/symptom recording continues to function independently.	H	H	✓
Performance	A user opens the cycle history screen (web or mobile).	Data retrieved and displayed without noticeable delay.	M	M	✓
Scalability	The number of active users increases over time.	Cloud backend & managed DB absorb growth without rework.	M	H	✓
Modifiability	A new symptom type is requested.	Added via the Symptom module without touching Cycle/Prediction.	M	L	—

Modifiability remains a valid candidate but is not carried into the Top 5 tactics slide — keeping Slide 5 aligned with the assignment's "top 5" requirement.

Top 5 ASRs & Architectural Tactics

1

Security

Stimulus: Unauthorized request for another user's health records

Tactic: Custom authentication & session validation on every request; authorization scoped to the requesting user.

Health data must never be retrievable by anyone but its owner.

2

Privacy

Stimulus: User requests deletion of their account

Tactic: Account-level deletion workflow that removes associated health records.

Supports user data rights; reduces retained sensitive data.

3

Availability

Stimulus: Prediction function becomes unavailable or slow

Tactic: Logical separation of Prediction & Analytics from the Cycle/Symptom recording path.

Users must log data even if insights are temporarily unavailable.

4

Performance

Stimulus: User opens the cycle history screen

Tactic: Efficient relational queries; minimized client-API round trips.

Cycle history is read frequently — query efficiency drives perceived speed.

5

Scalability

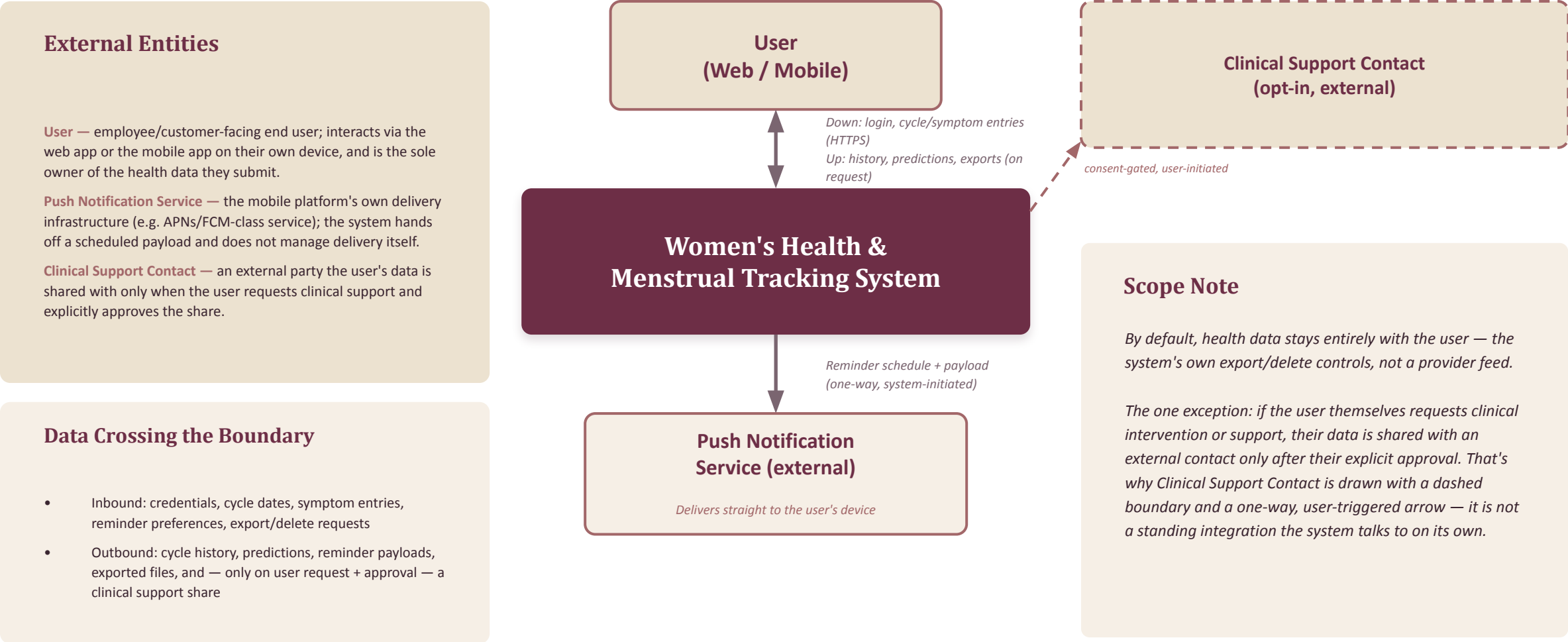
Stimulus: Active user base increases over time

Tactic: Reliance on the cloud provider's managed scaling for compute & DB tiers.

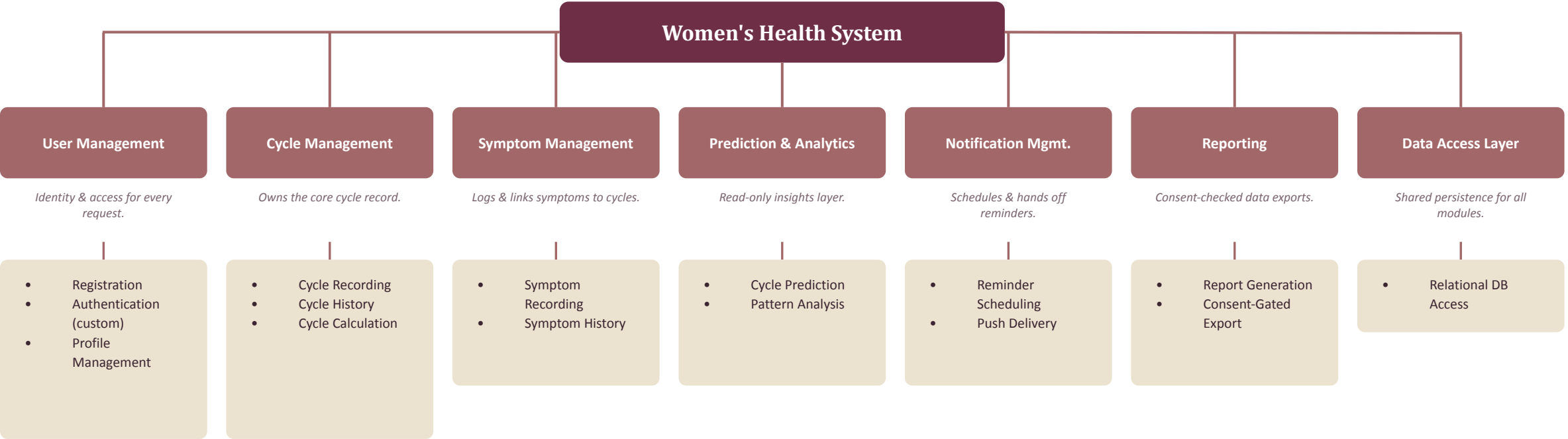
Matches the existing cloud-hosted deployment model.

Context Diagram

System boundary, external entities, and the data that crosses it — grounded in how the production system actually behaves.



Module Decomposition

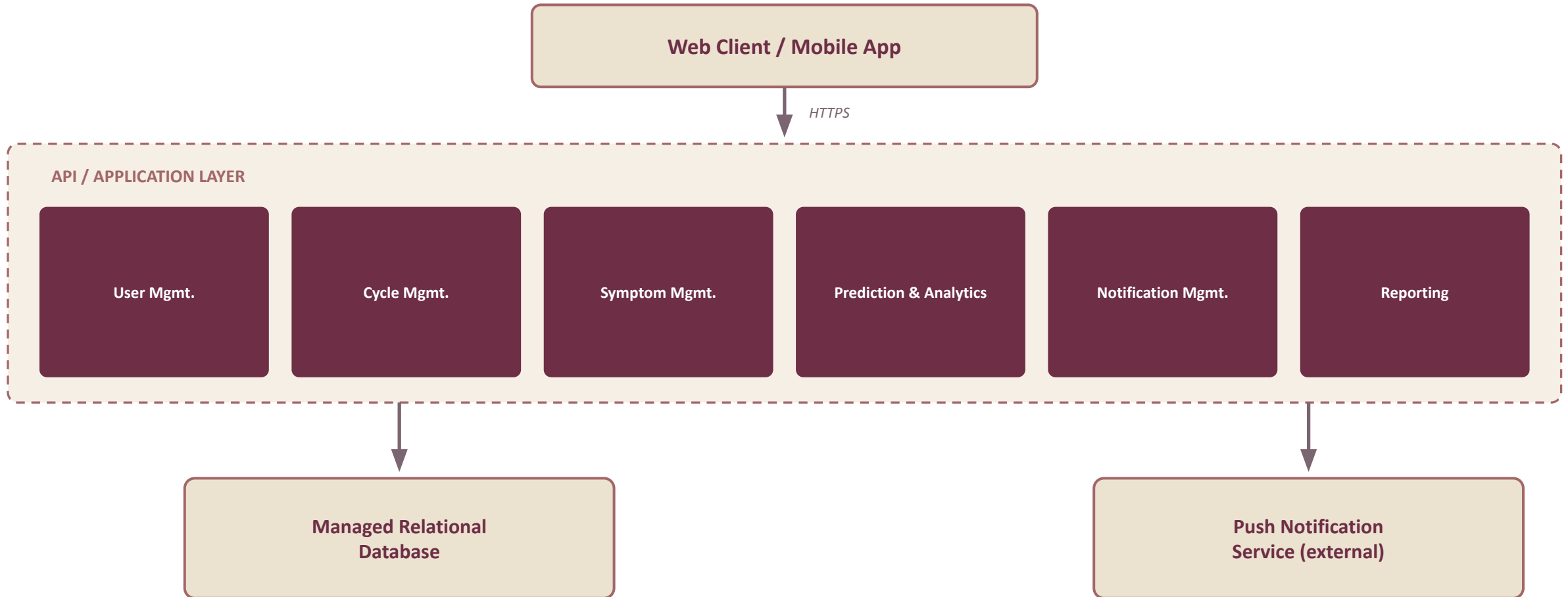


HOW THE MODULES RELATE

- User Management issues the session every other module trusts — nothing reaches Cycle, Symptom, or Reporting without it.
- Cycle & Symptom Management write through the Data Access Layer; Prediction & Analytics only reads from it, so a slow or failing prediction run can never corrupt recorded data.
- Notification Mgmt. and Reporting are the two modules that reach outside the application layer — to the push service and to the user's export, respectively — everything else stays internal.

Logical modules within the application layer — naming stays identical across the Component (Slide 8) and Deployment (Slide 9) diagrams.

Component & Connection Diagram

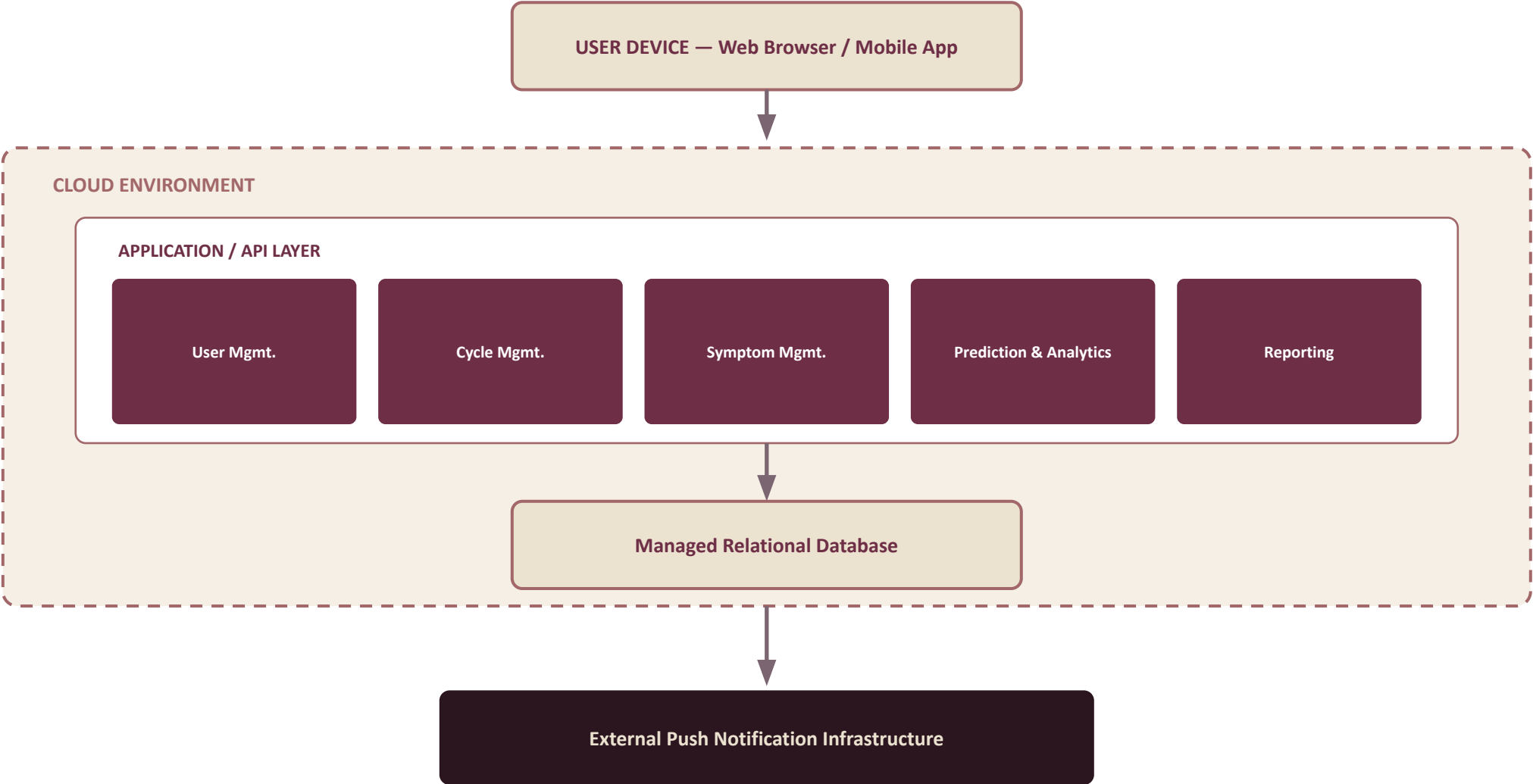


User, Cycle, Symptom, Prediction & Analytics, and Reporting all read/write through the Database. Notification Mgmt. is the one module that connects outward to the external push service.

Note: these are logical components in a single application layer, not independently deployed microservices — Cycle and Symptom logic may share code paths (modular monolith).

Deployment Diagram

Physical / deployment placement — distinct from the logical view in Slide 8.



All traffic HTTPS, DB backup & encryption-at-rest = platform defaults.

How the System Works



EXCEPTION HANDLING

Validation fails → Return error to user.

Insufficient history → Skip prediction; inform user more data is needed.

Push notification unavailable → Failure handled without affecting stored data; in-app fallback is a possible future improvement.

THEORY vs. WORKPLACE REALITY

Theoretical Architecture	Operational Reality
Fully independent microservices	Cycle & Symptom may share code paths (modular monolith).
Custom notification infrastructure	Delegated to the mobile platform's push service.
Bespoke encryption implementation	Managed database's built-in encryption at rest.

Key Learnings & Constraints

Key Learnings

- Architecture must be evaluated against quality attributes — claims must trace to the actual system, not idealized assumptions.
- Security and Privacy call for different tactics: authentication/authorization vs. a deletion workflow.
- A utility tree is only useful when stimuli/responses are concrete enough to prioritize — and should visibly narrow to the top 5.
- Logical modules and deployed services are not the same thing; a component diagram should state which it shows.
- Managed cloud services shift NFRs (durability, encryption, delivery) onto the platform — worth stating explicitly.
- Workplace systems rarely match idealized diagrams — naming where the real system diverges is part of a credible analysis.

Key Constraints

Security & Privacy — Depends on the custom auth layer — no third-party identity provider fallback.

Platform Dependency — Push delivery depends on external infra; in-app is the fallback.

Database Coupling — One managed relational DB across modules — schema changes have cross-module impact.

Cloud Provider Dependency — Scalability & availability bounded by the platform's config and limits.

Cost — Managed services trade custom control for reduced ops burden, at ongoing cost.

Future Improvements

- Introduce audit logging for access to health records
- Evaluate splitting Cycle & Symptom into independently deployable services
- Add a fallback in-app notification path
- Periodically review custom auth against current security practice